

matplotlib入門

1枚のグラフを「育てる」

rkskmt

1

matplotlibでグラフを描く

2

前回のおさらい

① まず可視化せよ

- 見ればわかる。見なければ見逃す — アンスコム、Datasaurus
- グラフは **ウソをつく** — y軸切断、期間チェリーピック
- ここまでは **読む側** の話。今日から **描く側** へ
 - 道具は **matplotlib**。1枚のグラフを「資料に載せられるレベル」まで育てる

3

matplotlib とは

- Pythonにおけるグラフ描画の **デファクトスタンダード**
 - **マットプロットリブ**と読む
- 内部はC実装で高速、NumPyの **ndarray** と相性が良い
 - seaborn や pandas の plotting 機能など、派生ライブラリも多い

ターミナルからインストール：

```
$ pip install matplotlib japanize-matplotlib
```

Shell

冒頭で import (**plt** としてインポートするのが一般的)：

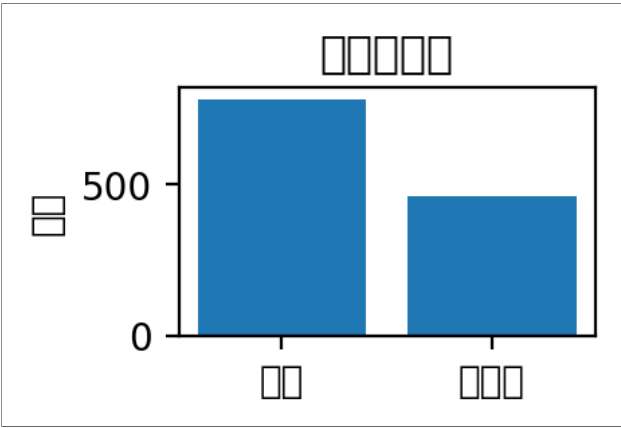
```
1 import matplotlib.pyplot as plt
2 import japanize_matplotlib
```

Python

4

文字化け対応

- matplotlib は多くの環境で **日本語が文字化け** する



- そこで **** import japanize_matplotlib ****
 - これだけで解決

⚠ rcParams でフォント指定しない

- **rcParams["font.family"]** は環境差で失敗しやすい
- 日本語表示は **japanize_matplotlib** に任せる

5

まずはドキュメントをチラ見

- 公式の **Plot types** ページに、描けるグラフの一覧がある
- https://matplotlib.org/stable/plot_types/index.html

💡 ライブラリとの接し方

- まず **何ができるかを大体把握**
- sandbox的な環境にインストールする
- sampleを動かしてTweak

6

東京の気温変化を図示する

- **気象庁の実測データ**：東京の月別平均気温
→ **1900年** と **2024年**、124年離れた2つの年
- 材料データ（以下をコピペ）：

```
1 import numpy as np
2
3 months = np.arange(1, 13)    # 1~12月
4 temp_1900 = np.array([1.6, 3.1, 5.7, 11.4, 17.3, 19.3, 22.8, 26.1, 22.6, 16.5, 11.0, 5.5])
5 temp_2024 = np.array([7.1, 8.0, 9.6, 17.1, 20.0, 23.1, 28.7, 29.0, 26.6, 20.6, 13.7, 8.1])
```

① 出典

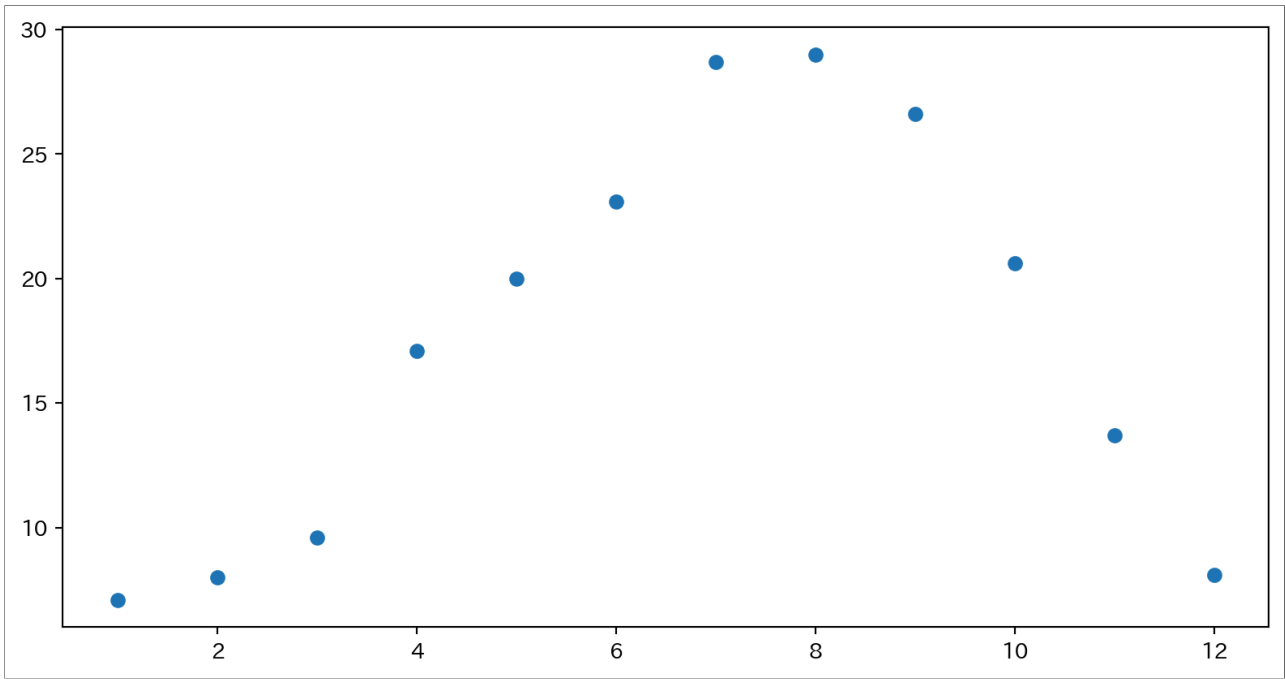
気象庁「過去の気象データ検索」東京（月平均気温）

- **ここから グラフを育てていく**

7

まずは点のプロット

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import japanize_matplotlib
4
5 months = np.arange(1, 13)
6 temp_2024 = np.array([7.1, 8.0, 9.6, 17.1, 20.0, 23.1, 28.7, 29.0, 26.6, 20.6, 13.7, 8.1])
7
8 plt.plot(months, temp_2024, 'o')    # 'o' で点だけプロット
9 plt.show()
```

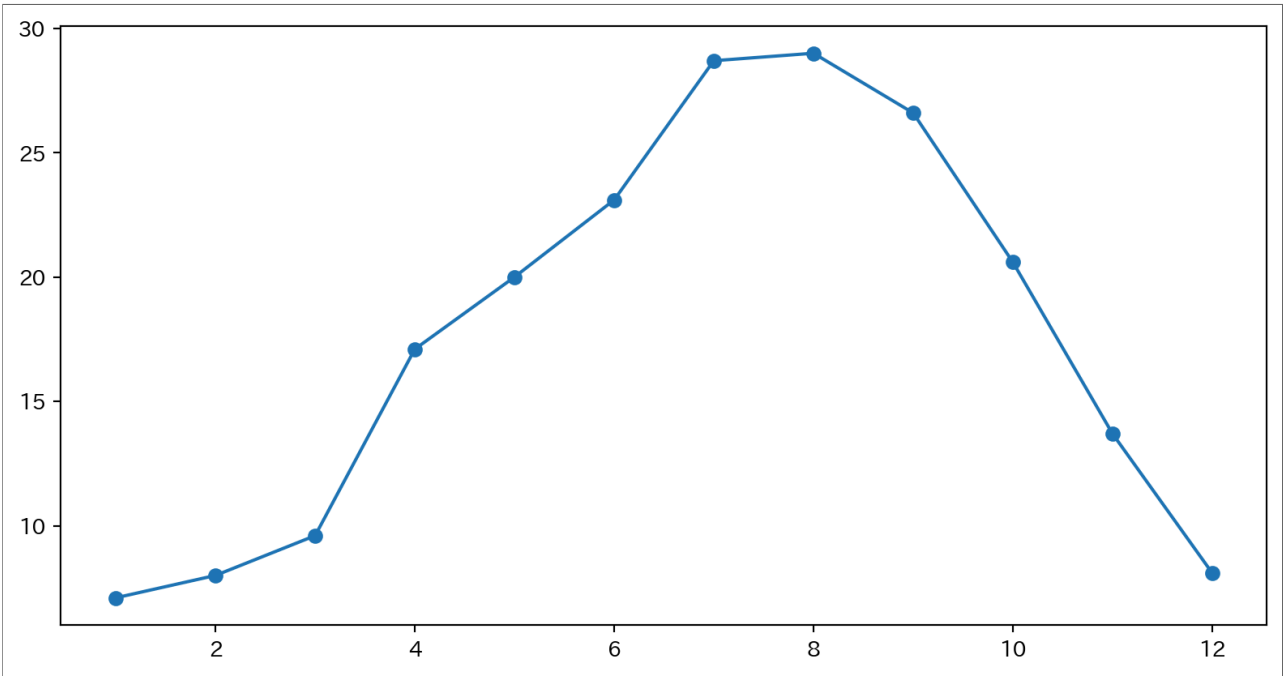


- 一番基本の **plt.plot(x, y, 'o')** — **'o'** は **マーカー** を表す
→ **x** も **y** も **ndarray** や **list** でOK
- **季節の山がもう見えている**

8

点に線を付け足す

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import japanize_matplotlib
4
5 months = np.arange(1, 13)
6 temp_2024 = np.array([7.1, 8.0, 9.6, 17.1, 20.0, 23.1, 28.7, 29.0, 26.6, 20.6, 13.7, 8.1])
7
8 plt.plot(months, temp_2024, 'o-') # 点+線
9 plt.show()
```



- 第3引数を **省略すると折れ線**
- **'o-'** だと **点+線**
- 月ごとの **つながり**（連続的な変化）が見えるようになった

Tweak：線の見た目



- さっきのコード の **'o-'** を以下に **書き換えて実行** せよ

やってみる	'o-' をこう変える
点を消して線だけにする	'-'
線を消して点だけにする	'o'
線を破線にする	'--'

💡 ここで質問

- **一点鎖線**（**-・-・-**）にするには、どう書くと思う？

➡ 答え：線の早見表

演習：二次関数を描く



- $y = x^2 - 2x + 3$ を **定義域** $-2 \leq x \leq 5$ で描け
- 手順
 1. **np.linspace(-2, 5, 200)** で **x** を作る（細かく刻むと線が滑らかになる）
 2. **y** を計算（**ndarray** はそのまま式に入れられる — NumPy回の復習）
 3. **plt.plot(x, y)** で折れ線として描く

💡 平方完成で検算

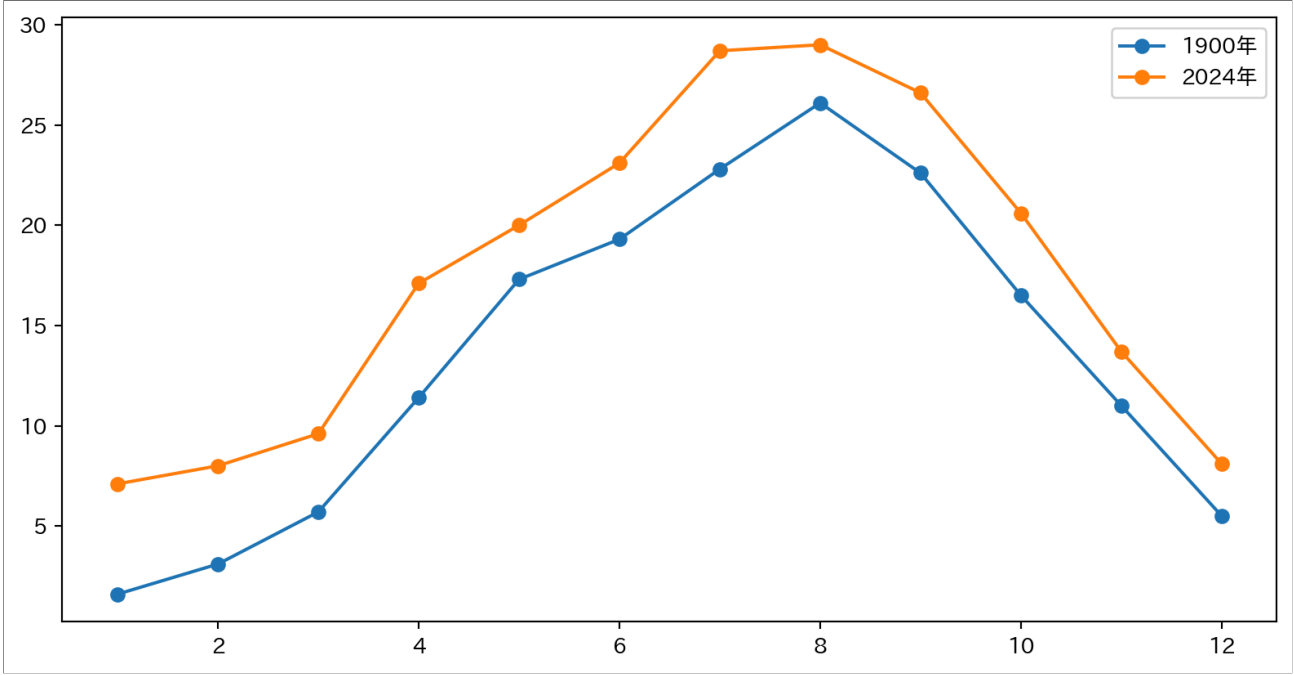
- $y = (x - 1)^2 + 2$ なので **頂点は (1, 2)**
- グラフの谷が **(1, 2)** に来ているか確認

▶ 解答例を見る

2本目を重ねる・凡例をつける

- plt.plot() を 2回呼ぶと重ね描き になる
 - どの線が何かを示すために 凡例 も書ける
- plot関数に引数：label=
- その後plt.legend() を呼ぶ

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import japanize_matplotlib
4
5 months = np.arange(1, 13)
6 temp_1900 = np.array([1.6, 3.1, 5.7, 11.4, 17.3, 19.3, 22.8, 26.1, 22.6, 16.5, 11.0, 5.5])
7 temp_2024 = np.array([7.1, 8.0, 9.6, 17.1, 20.0, 23.1, 28.7, 29.0, 26.6, 20.6, 13.7, 8.1])
8
9 plt.plot(months, temp_1900, 'o-', label="1900年") # 凡例ラベル
10 plt.plot(months, temp_2024, 'o-', label="2024年")
11 plt.legend() # 凡例を表示
12 plt.show()
```



- 重ねた瞬間、124年分の差 が一目で見える
- 2024年の冬は1900年の春なみ、夏は ほぼ全月で上

Tweak：凡例



- 前のスライドのコード の以下の部分を変えよ

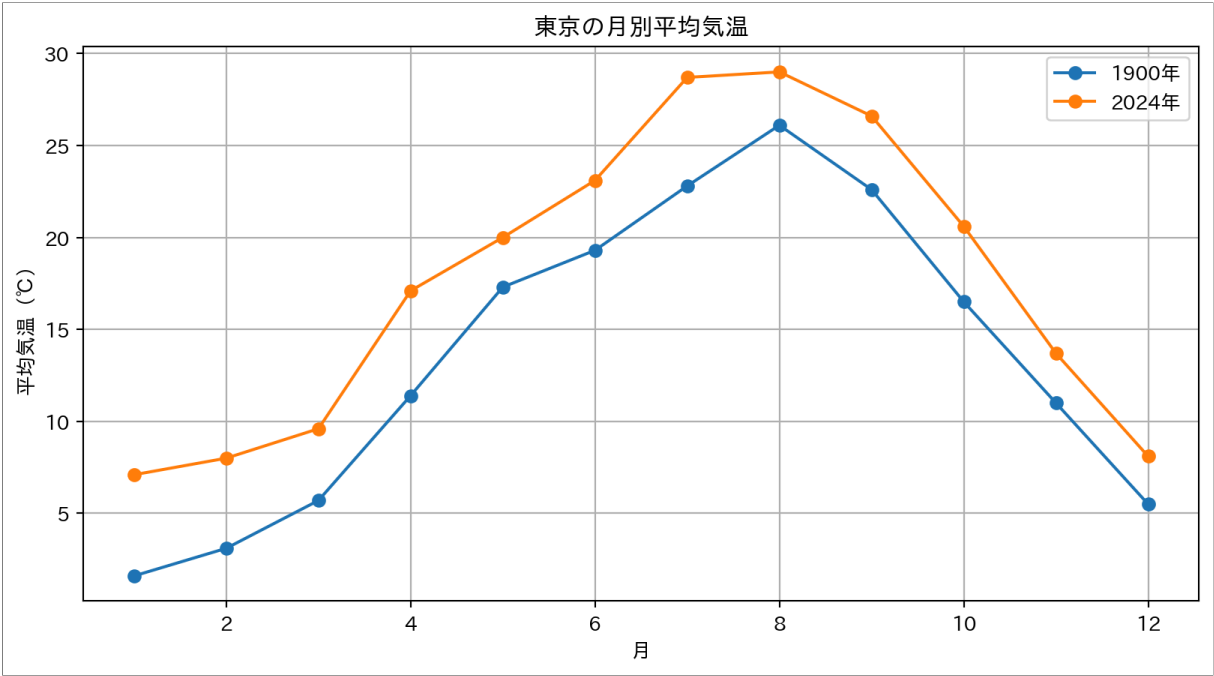
やってみる	どう変える
凡例の文字を変える	label="1900年" → label="明治33年"
片方を凡例から消す	一方の label=... だけ消す
凡例ごと消す	plt.legend() の行を消す

💡 予想してから試す

- plt.legend() を 2本の plt.plot より前の行 に動かすと、凡例はどうなる？
- まず予想 → 実行して確かめる（順番が関係する？）

- `plt.xlabel()` / `plt.ylabel()` で 軸ラベル
- `plt.title()` で グラフタイトル
- `plt.grid(True)` で グリッド線

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import japanize_matplotlib
4
5 months = np.arange(1, 13)
6 temp_1900 = np.array([1.6, 3.1, 5.7, 11.4, 17.3, 19.3, 22.8, 26.1, 22.6, 16.5, 11.0, 5.5])
7 temp_2024 = np.array([7.1, 8.0, 9.6, 17.1, 20.0, 23.1, 28.7, 29.0, 26.6, 20.6, 13.7, 8.1])
8
9 plt.plot(months, temp_1900, 'o-', label="1900年")
10 plt.plot(months, temp_2024, 'o-', label="2024年")
11 plt.legend()
12 plt.title("東京の月別平均気温")           # タイトル
13 plt.xlabel("月")                           # x軸ラベル
14 plt.ylabel("平均気温 (°C) ")              # y軸ラベル
15 plt.grid(True)                             # グリッド線
16 plt.show()
```

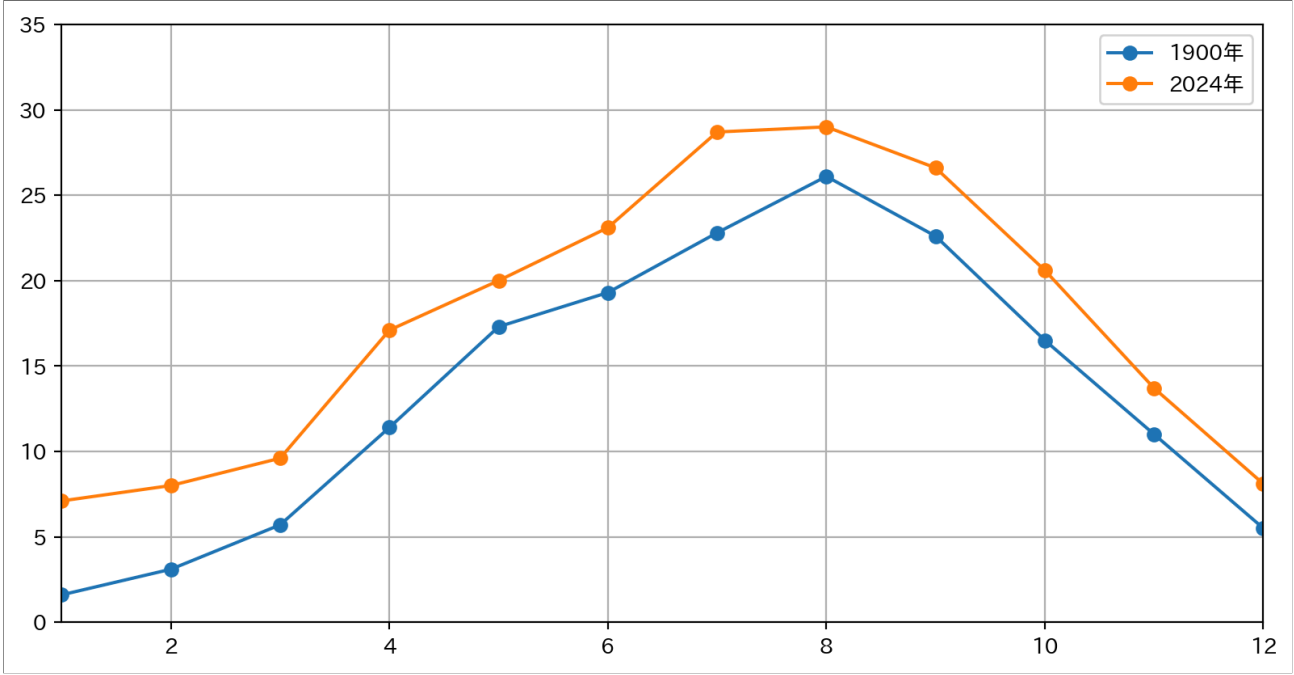


- 💡 装飾の優先順位
- 軸ラベル (何の量か)
 - 凡例 (どの線が何か)
 - タイトル (何のグラフか)
 - この3つが揃っていない図は 資料に載せてはいけない

表示範囲の指定 (xlim / ylim)

- plt.xlim(a, b) / plt.ylim(a, b) で軸の範囲を固定
- 何も指定しないと **データの範囲+少し余白** が自動で選ばれる

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import japanize_matplotlib
4
5 months = np.arange(1, 13)
6 temp_1900 = np.array([1.6, 3.1, 5.7, 11.4, 17.3, 19.3, 22.8, 26.1, 22.6, 16.5, 11.0, 5.5])
7 temp_2024 = np.array([7.1, 8.0, 9.6, 17.1, 20.0, 23.1, 28.7, 29.0, 26.6, 20.6, 13.7, 8.1])
8
9 plt.plot(months, temp_1900, 'o-', label="1900年")
10 plt.plot(months, temp_2024, 'o-', label="2024年")
11 plt.legend()
12 plt.xlim(1, 12)      # x軸: 1~12月
13 plt.ylim(0, 35)      # y軸: あえて0から見せる
14 plt.grid(True)
15 plt.show()
```



⚠ **y軸の罫 (描く側の責任)**

- 範囲をデータに合わせると 小さな変化が大きく見える
- 比較するグラフは **y軸を揃える** か **0から見せる** のが原則

Tweak：軸の範囲で印象を操る



- さっきのコード の軸だけ変えて実行 — 同じデータでも印象がどう動くか

やってみる	どう変える
差を誇張する	<code>plt.ylim(0, 35)</code> → <code>plt.ylim(15, 30)</code>
夏だけ切り取る	<code>plt.xlim(1, 12)</code> → <code>plt.xlim(6, 9)</code>

💡 予想してから試す

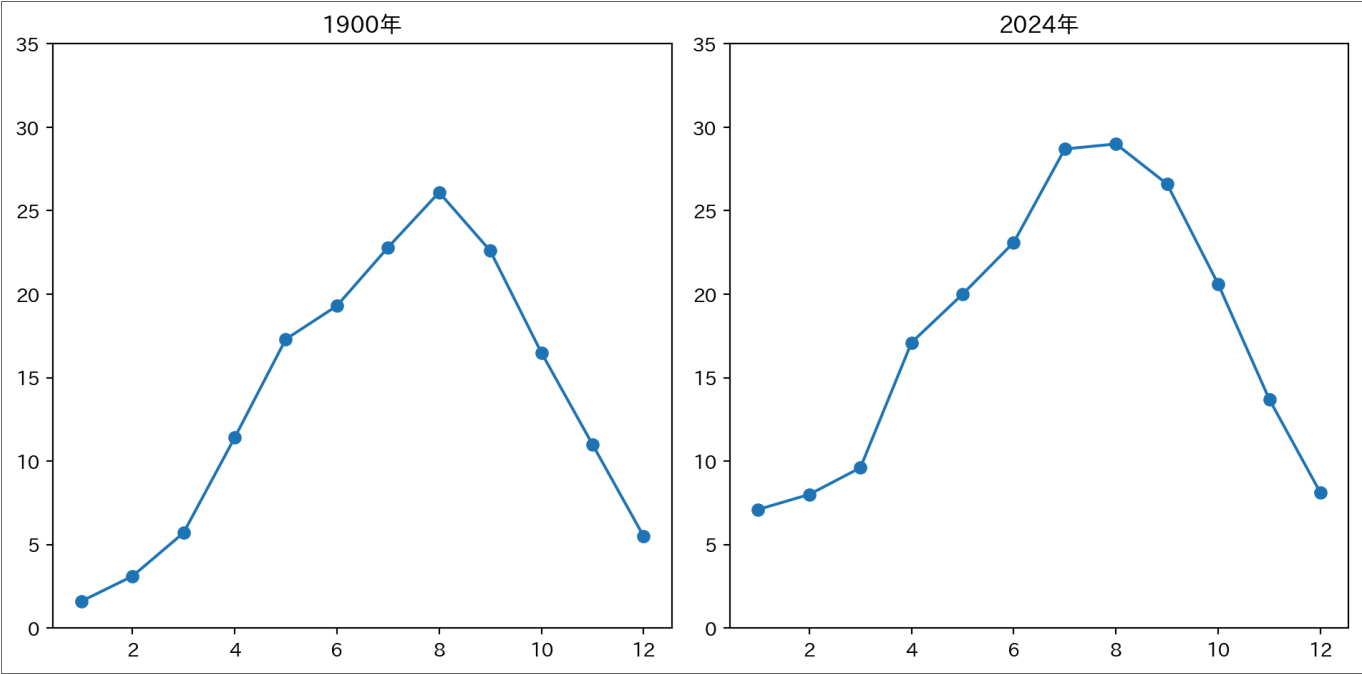
- `plt.ylim(...)` の行を **消す** (自動範囲) と、y軸の下限は 0 になる？

複数のグラフを並べる (subplot)

- `plt.subplot(行数, 列数, 場所番号)` で基盤の目状に並べる
- 場所番号は **1**始まり、左上から右へ
- 例：`plt.subplot(2, 2, 3)` は2×2の3番目（左下）

1 | 2
+
3 | 4

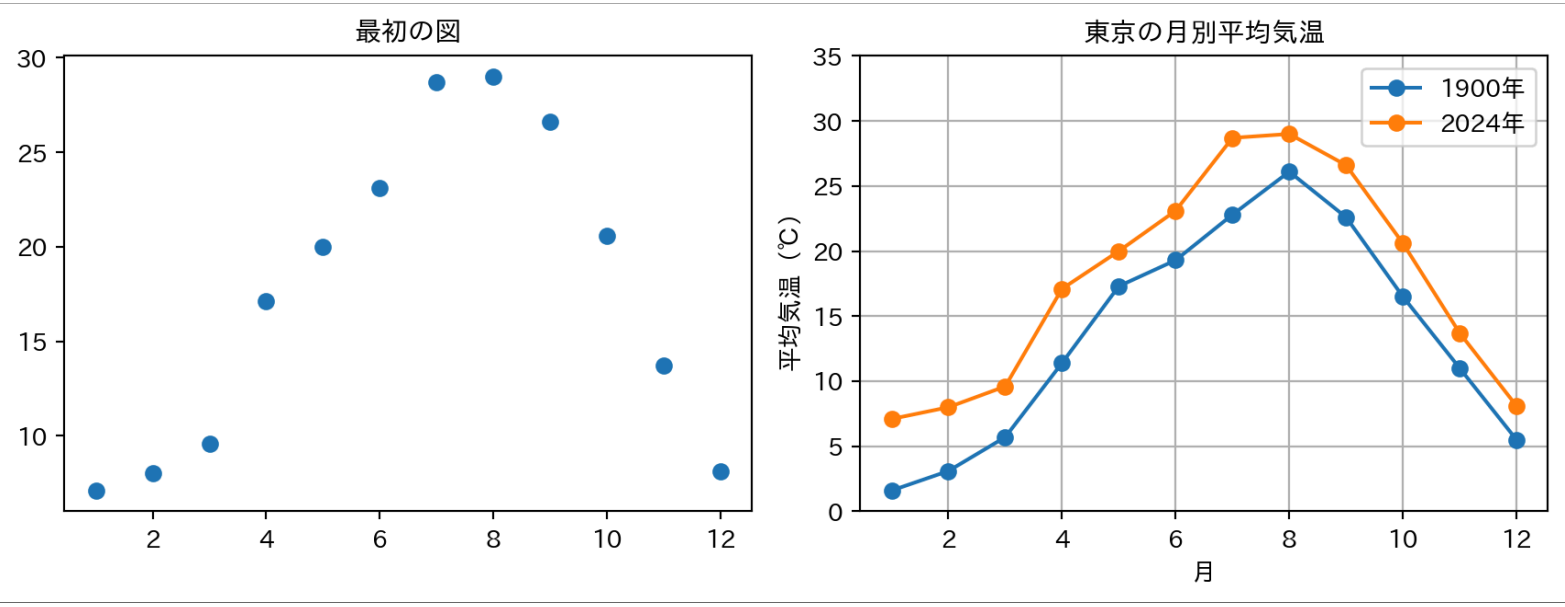
```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import japanize_matplotlib
4
5 months = np.arange(1, 13)
6 temp_1900 = np.array([1.6, 3.1, 5.7, 11.4, 17.3, 19.3, 22.8, 26.1, 22.6, 16.5, 11.0, 5.5])
7 temp_2024 = np.array([7.1, 8.0, 9.6, 17.1, 20.0, 23.1, 28.7, 29.0, 26.6, 20.6, 13.7, 8.1])
8
9 plt.subplot(1, 2, 1)      # 1行2列の左
10 plt.plot(months, temp_1900, 'o-')
11 plt.ylim(0, 35)          # 並べて比較するなら軸を揃える
12 plt.title("1900年")
13
14 plt.subplot(1, 2, 2)      # 1行2列の右
15 plt.plot(months, temp_2024, 'o-')
16 plt.ylim(0, 35)
17 plt.title("2024年")
18
19 plt.tight_layout()        # 余白を自動調整
20 plt.show()
```



i 重ねる vs 並べる

- 直接比較したい → **重ねる**（さっきの凡例つき1枚）
- 形をそれぞれ見たい・線が多すぎる → **並べる**（subplot）

ここまでの成果：図は「育てる」

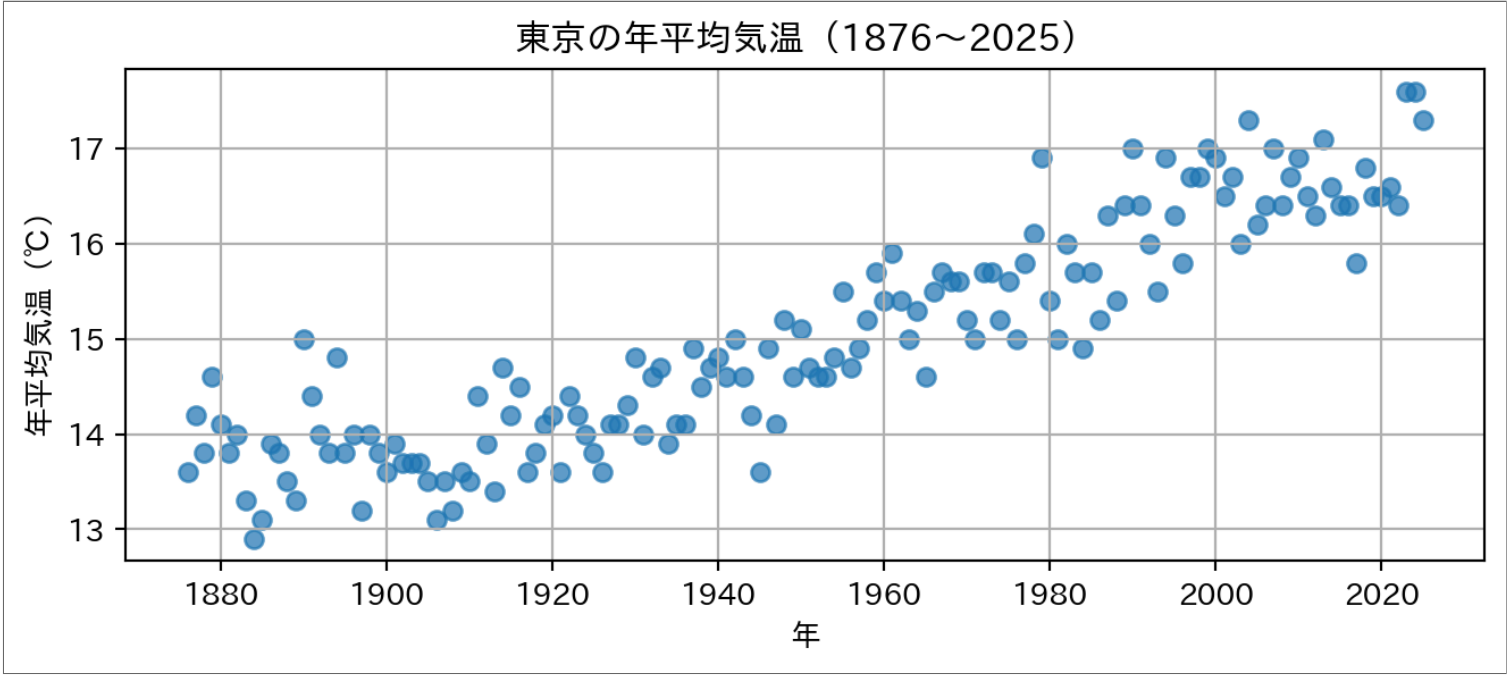


- **同じデータ**。でも伝わり方は別物
- 育てた手順：点 → 線 → 重ねる＋凡例 → ラベル・タイトル → 軸の範囲

散布図で150年を一気に見る

- 年平均気温150年分のCSV（気象庁）を `np.loadtxt` で読み込む
→ データ：[tokyo_temp_annual.csv](#)（`data` フォルダを作成して中に置く）
- 大量の点は `plt.scatter()`
→ `alpha` で 透明度 を指定すると重なりが見やすい

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import japanize_matplotlib
4
5 data = np.loadtxt("data/tokyo_temp_annual.csv", delimiter=",", skiprows=1)
6 year = data[:, 0]      # 1列目：年
7 temp = data[:, 1]      # 2列目：年平均気温
8
9 plt.figure(figsize=(7, 3.2))  # 横長・低めにしてスライドに収める
10 plt.scatter(year, temp, alpha=0.7)
11 plt.xlabel("年")
12 plt.ylabel("年平均気温 (°C) ")
13 plt.title("東京の年平均気温 (1876~2025) ")
14 plt.grid(True)
15 plt.tight_layout()
16 plt.show()
```



結論：東京は暑くなった。ただし



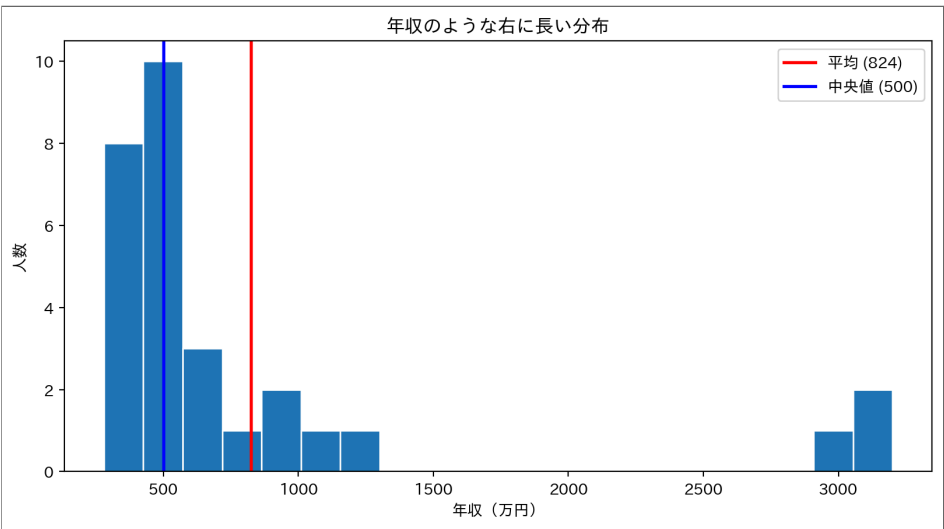
- 150年で 約4°C上昇
- この上昇には 都市化（ヒートアイランド） の影響も大きく含まれる
- 図は答えをくれるが、 解釈には背景知識が要る

ヒストグラム：所得分布の可視化

- 前回の厚労省の所得分布は ヒストグラム（区間ごとに数えるグラフ）だった
- 同じような構造の図を、ある会社の社員29人の年収で再現したとする

→ `plt.hist(x, bins=区間の数)` で描ける
→ `plt.axvline(値)` で 縦線、`np.median(x)` で 中央値

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import japanize_matplotlib
4
5 income = np.array([
6     280, 310, 330, 350, 360, 380, 400, 410, 430, 450, 470, 500, 520, 450, 470, 500, 520, 550, 600, 650, 700, 760, 900, 950, 1100
7 ])
8
9 mean_income = income.mean()
10 median_income = np.median(income)
11
12 plt.hist(income, bins=20, edgecolor="white")
13 plt.axvline(mean_income, color="red", linewidth=2, label=f"平均 ({mean_income:.0f})")
14 plt.axvline(median_income, color="blue", linewidth=2, label=f"中央値 ({median_income:.0f})")
15 plt.xlabel("年収 (万円) ")
16 plt.ylabel("人数")
17 plt.title("年収のような右に長い分布")
18 plt.legend()
19 plt.show()
```



① 「平均年収824万円の会社」

- この会社は求人票に「平均年収824万円」と書かれて、そのくらいもらえると錯覚する
- でも中央値は **500万円** — 半分の人は500万以下
- 描けば誰でも分かる

Tweak：ビンの数で分布が化ける



- さっきのコード の `bins` だけ変えて実行 — 同じ29人なのに形が変わる

やってみる	どう変える
粗く刻む	<code>bins=20</code> → <code>bins=5</code>
細かく刻む	<code>bins=20</code> → <code>bins=50</code>

💡 予想してから試す

- `bins=5` にすると、右に長く伸びる **高年収の外れ値** の山は見える？消える？
- まず予想 → 実行して確かめる（軸の範囲と同じで、ビンの切り方も印象を動かす）

plt早見表

操作	コード
点だけ	<code>plt.plot(x, y, 'o')</code>
折れ線	<code>plt.plot(x, y)</code>
棒グラフ	<code>plt.bar(x, y)</code>
散布図	<code>plt.scatter(x, y)</code>
ヒストグラム	<code>plt.hist(x)</code>
縦線	<code>plt.axvline(x)</code>
軸の範囲	<code>plt.xlim(a, b)</code> / <code>plt.ylim(a, b)</code>
軸ラベル	<code>plt.xlabel("...")</code> / <code>plt.ylabel("...")</code>
タイトル	<code>plt.title("...")</code>
凡例	<code>plt.legend()</code> (要 <code>label=...</code>)
グリッド	<code>plt.grid(True)</code>
並べる	<code>plt.subplot(行, 列, 番号)</code>
CSV読み込み	<code>np.loadtxt("file.csv", delimiter=",", skiprows=1)</code>

23

演習：謎のデータをプロットせよ



- [datasaurus_star.csv](#) をダウンロードして、`data/` フォルダに置く
- 前回見た [Datasaurus Dozen 13個のうちの1つ](#) (恐竜ではない)
→ 統計量はもちろん： $\bar{x} = 54.27$ 、 $\bar{y} = 47.84$ 、 $r = -0.06$
- 手順
 1. `np.loadtxt` で読み込む (1行目はヘッダなので `skiprows=1`)
 2. `data[:, 0]` と `data[:, 1]` で `x`, `y` を取り出す
 3. `plt.scatter` でプロット — 何が見える？
 4. 軸ラベル・タイトルをつける

▶ 解答例を見る

24

演習：アンスコム の四重奏を再現



- 手順
 1. 下のデータをコピペして使う
 2. `plt.subplot(2, 2, i)` で2×2に並べる
 3. それぞれ `scatter` でプロット
 4. `np.polyfit(x, y, 1)` で1次回帰直線を求め、`plot` で重ねる
 5. すべてのサブプロットで `xlim` / `ylim` を揃える

```
1 import numpy as np
2
3 # データセット I, II, III (x は共通)
4 x1 = np.array([10, 8, 13, 9, 11, 14, 6, 4, 12, 7, 5])
5 y1 = np.array([8.04, 6.95, 7.58, 8.81, 8.33, 9.96, 7.24, 4.26, 10.84, 4.82, 5.68])
6 y2 = np.array([9.14, 8.14, 8.74, 8.77, 9.26, 8.10, 6.13, 3.10, 9.13, 7.26, 4.74])
7 y3 = np.array([7.46, 6.77, 12.74, 7.11, 7.81, 8.84, 6.08, 5.39, 8.15, 6.42, 5.73])
8
9 # データセット IV (x が異なる)
10 x4 = np.array([8, 8, 8, 8, 8, 8, 8, 19, 8, 8, 8])
11 y4 = np.array([6.58, 5.76, 7.71, 8.84, 8.47, 7.04, 5.25, 12.50, 5.56, 7.91, 6.89])
```

① なぜ軸を揃える？

- 4つのプロットを 公平に比較 するため
- 違うスケールで並べると印象操作になる

▶ 解答例を見る

25

演習：好きな画像を表示



- matplotlib は **画像も描画**できる
- 公式チュートリアルを参考
- <https://matplotlib.org/stable/tutorials/images.html>

```
1 import matplotlib.pyplot as plt
2 import japanize_matplotlib
3 import matplotlib.image as mpimg
4
5 img = mpimg.imread("something.jpg") # 画像名。フォルダの場合はpathを指定
6 plt.imshow(img)
7 plt.axis("off")           # 軸を消す
8 plt.show()
```

まとめ

- **matplotlib**：「東京は暑くなったのか」を1枚のグラフに育てた
→| 点 → 線 → 重ねる+凡例 → ラベル・タイトル → 軸の範囲
→| 散布図で150年、ヒストグラムで分布
- 装飾の優先順位：**軸ラベル・凡例・タイトル** — 揃っていない図は資料に載せない
- **可視化編の謎**（1日だけの大きな山）は **API編で回収する**

参考リンク

- [Matplotlib “Pyplot Tutorial”](#)
- [Matplotlib “Plot types”](#)
- *Python Data Science Handbook* — Chapter 3 Visualizations
- [Same Stats, Different Graphs — Datasaurus Dozen \(Autodesk Research\)](#)
- [気象庁「過去の気象データ検索」](#)

付録：線とマーカーの早見表

- **plt.plot(x, y, 'o-')** の第3引数は **マーカー・線種・色** を文字で並べたもの
- 暗記する必要はない。「こう変えるとこうなる」を一度試せば十分

線種 (line style)	記号
実線	-
破線	--
一点鎖線	-.
点線	:
マーカー (marker)	記号
丸	o
四角	s
三角	^
星	*

- 組み合わせ例：**'o--'**（丸+破線）・**'s:'**（四角+点線）・**'r-.'**（赤の一点鎖線）